

Summary of LLM Inference 2/24/2026

Blake Potvin (potvinbl@umich.edu), Toren Mwindare (torencem@umich.edu), Ethan Beaird (beaird@umich.edu), Kidus Simegne (kidus@umich.edu)

LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism

Problem and Motivation

As LLM context windows continue to grow, inference workloads become increasingly variable across requests and across phases of execution. Larger context windows enable longer prompts, retrieved documents, and conversation histories, but they also produce wider variation in request lengths and runtime cost. In particular, the prefill phase (which processes the input prompt) and the decoding phase (which generates output tokens iteratively) impose different computational, memory, and communication demands.

Despite this, many existing serving systems still rely on static parallelism strategies, where the number of GPUs and the way computation is partitioned are decided in advance and remain unchanged during execution. As request lengths vary across requests and resource demands shift between prefill and decoding, this fixed configuration can become poorly matched to the workload, leading to inefficient resource utilization, higher communication overhead, and lower throughput. This mismatch motivates the authors' pursuit of more adaptive serving mechanisms that can respond to runtime variability rather than assuming a uniform workload. More broadly, improving LLM serving performance increasingly depends on parallelism strategies that can adapt to changing request characteristics and phase behavior.

Related Works

Prior work on LLM serving has focused on improving inference efficiency through optimizing GPU execution and parallelism. At the single GPU level, techniques such as FlashAttention and FlashDecoding reduce attention computation overhead and improve kernel efficiency. At the multi-GPU level, serving systems commonly use model-parallel approaches such as tensor parallelism to partition model parameters across devices and accelerate computation. While these techniques are effective, as mentioned earlier, they typically require the degree of parallelism to be fixed before deployment, which limits their ability to adapt to changing workload conditions at runtime.

To better support long-context requests, existing serving systems also introduced strategies such as chunked prefill and prefill-decoding disaggregation. Chunked prefill is an LLM serving technique that splits the prompt processing stage (prefill) of a long request into smaller token chunks, and processes those chunks across multiple forward steps instead of all at once. This reduces the peak resource pressure of long prompts and allows the system to better schedule prefill work alongside decoding requests, rather than letting one very long prefill

monopolize the GPU. Prefill-decoding disaggregation also addresses this interference by assigning prefill and decoding to separate GPU groups. However, this separation introduces new tradeoffs, including communication overhead from frequent KV cache migration, inefficient GPU utilization when static group assignments do not match runtime demand, and memory fragmentation caused by isolation across groups.

Sequence parallelism, developed for LLM training, offers another relevant direction. Sequence parallelism splits the computation for a long sequence across multiple GPUs. Because each device only handles part of the sequence at a time, it does not need to store as many intermediate activations/buffers, which can reduce memory pressure. It is also compatible with common attention designs (like MHA, MQA, and GQA) and can be combined with other forms of parallelism, including tensor parallelism. However, sequence parallelism based approaches are designed for training settings with fixed parallelism configurations and do not directly address the requirements of online serving. In particular, they do not naturally support the decoding phase, dynamic inference traffic, or efficient KV-cache management under changing runtime conditions.

All these lines of work substantially improve LLM inference performance, but they are largely static in regard to how they allocate and organize parallelism. This creates a gap for serving systems that must handle highly variable request lengths and phase-dependent resource demands, motivating the work of the authors.

Solution Overview

LoongServe is an LLM serving system for long-context inference built around Elastic Sequence Parallelism (ESP), which dynamically adjusts the degree of parallelism (DoP) at iteration granularity according to request length and inference phase. Rather than fixing a static parallel configuration or statically partitioning GPUs into phase-specific groups, LoongServe organizes elastic instances into disjoint ESP groups that can be reconfigured online to better match the highly variable compute and memory demands of long-context, variable-length workloads.

One challenge in ESP is that changing DoP can require redistributing KV cache state across instances, which can dominate latency for long contexts. LoongServe addresses this with a proactive scale-down mechanism in the prefill phase: rather than performing reactive KV migration after prefill, it reuses the KV circulation already present in sequence parallelism and selectively retains KV tensors on the target post-scale-down-group during prefill, thereby avoiding additional migration communication for that transition. For decoding, the system introduces multi-master distributed decoding; multiple master instances each own different requests in the batch, enabling distributed decoding without migrating existing KV caches during scale up, reducing memory fragmentation, and improving compute efficiency by parallelizing local-layer work across masters while overlapping communication with local attention computation.

To support online adaptation, LoongServe uses a global manager (with profiling data from the scaling information base) that makes iteration-level scheduling decisions over request dispatch, elastic instance allocation, batching/DoP decisions, KV-cache placement, and elastic scaling plans. It decomposes scheduling into a four-step algorithm (dispatching, elastic instance

allocation, batching, and elastic scaling plan generation) with polynomial-time substeps. The system also exposes a unified distributed KV cache pool, allowing token-granularity KV placement across elastic instances, which helps mitigate GPU memory fragmentation under variable-length workloads.

Limitations

LoongServe substantially improves adaptability and throughput for long-context serving, but its runtime decisions rely on an analytical performance model and profiling data, along with heuristic gain-loss comparisons in multiple scheduler stages. Accordingly, performance may depend on how well those model assumptions transfer across different hardware interconnects, model architectures, and distributions. In practice, deployment may still require nontrivial calibration. The scheduler is also explicitly a decomposed approximation, not a globally optimal optimizer. This design trades off optimality guarantees for online responsiveness.

Future Research Directions

The authors do not explicitly discuss future directions; any extensions would be reader inferences rather than stated claims.

A logical next step is online adaptation of the profiling data and analytical cost model, so scheduling decisions can continuously recalibrate under workload drift (e.g. changing request-length distributions, decode lengths, etc.) instead of depending primarily on offline profiling. This would make ESP more robust across deployments and time-varying traffic. Future work might also explore topology-aware and heterogeneous-cluster scheduling as well as integration with other inference optimizations (e.g. speculative decoding, KV compression, etc.).

MegaScale-Infer: Serving Mixture-of-Experts at Scale with Disaggregated Expert Parallelism

Problem and Motivation

The Effectiveness of LLMs is highly tied to the number of model parameters. As a matter of fact, there is a growing demand to scale up these models, i.e the number of model parameters. Due to their sparse activation architecture, Mixture of Experts (MoEs) became a very popular choice to meet the previously stated scaling demand. In MoE models, a gating network with trainable parameters routes input tokens to a subset of these experts (rather than engaging all the Feed Forward Networks/ FFNs) based on matrix multiplication results between the tokens embedding vectors and the trainable gating parameters. The sparse nature of this Model of Experts architecture allows for scaling the model size by increasing the number of experts without raising computational costs. For instance, Mixtral 8x22B has around 141B parameters, but its active parameters for each token are only approximately 39B with top-2 expert selection.

However, in MoEs architectures, larger model sizes are often accompanied by more experts and hence higher sparsity since a single expert will be assigned significantly less tokens within the same batch size as compared to traditional LLM inference. This increasing sparsity reduces the GPU utilization of FFN modules, making them no longer compute-intensive and thereby introducing unnecessary computational cost. This challenge motivates the disaggregation of attention from the LLM inference pipeline and the replication of attention modules to increase the decoding batch size for FFN modules, enabling them to operate more efficiently and increasing GPU utilization. Most previous works focus on optimizing dense model inference in long-context scenarios, making their methods less effective for MoE models, as they address memory capacity limitations rather than sparsity constraints. This gap calls for tailored approaches specifically designed for MoE inference.

Related Works

Prior works on LLM inference primarily focus on optimizing either the underlying hardware infrastructure or the serving system. Tools offering hardware-optimized inference such as *TensorRT* represent one of the most widely adopted frameworks for accelerating neural network inference on GPUs. TensorRT performs several types of optimizations to accelerate deep learning inference, including precision calibration (reducing numerical precision using INT8, FP16 and other formats to speed up computation), layer fusion to reduce memory access and computational overhead and automatic kernel tuning.

However, these optimizations are primarily designed for dense neural networks and do not fully address the challenges posed by sparse architectures. Sparse neural networks, which rely on weight pruning or structured sparsity to improve efficiency, often fail to achieve proportional performance gains under existing hardware-optimized frameworks. Although limited

support for sparse tensors has been introduced in *TensorRT*, the resulting acceleration remains less significant compared to dense models, leaving substantial optimization opportunities unexplored.

Tools like *vLLM* shifts optimization from kernel execution toward serving architecture design. The system introduces *PagedAttention*, a memory-virtualization mechanism that manages KV caches using pageable blocks rather than contiguous allocations. This design dramatically reduces memory fragmentation and enables continuous batching across heterogeneous user requests.

Both *vLLM* and *TensorRT* largely assume homogenous inference hardware architecture. They do not natively support and/or is not easy to incorporate heterogeneous deployment where different hardware resources assume specialized roles or where inference components such as attention computation and expert execution across the datacenter nodes. In contrast to these, *MegaScale Infer* enables heterogeneous deployment strategy of attention and FFN modules on different GPUs to fully leverage their different capabilities and achieve lower costs. For example, attention modules can be deployed on GPUs with more cost-effective memory capacity and bandwidth, while FFN modules can utilize GPUs with more affordable compute capability.

Solution Overview

Following the idea of disaggregating attention and FFN (i.e. expert) modules during inference, *MegaScale-Infer* deploys these components across different GPU groups instead of colocating them on the same devices. Attention modules are able to be replicated using data parallelism to process decoding requests concurrently, while expert modules are distributed across different GPUs (i.e. expert parallelism). After attention has been calculated and the gating decisions, the attention nodes route the tokens to expert nodes based on a top-k selection strategy. Because tokens from multiple replicas are aggregated at the expert nodes, the effective batch size per expert increases. In traditional MoE architectures, each expert only processes a fraction of the decoding batch, which reduces the compute intensity and makes them typically memory-intensive. By aggregating the tokens across replicas, *MegaScale-Infer* enables larger matrix multiplications back at the expert nodes which shifts executions to be compute-intensive and significantly increases GPU utilization.

This disaggregation introduces cross-node communication and can cause severe GPU idle time if this were executed sequentially. *MegaScale-Infer* addresses this in ping-pong pipeline parallelism, a strategy that breaks apart a batch into microbatches in order to efficiently interleave microbatches staggered across attention and expert nodes in pipeline fashion. This strategy is similar to pipeline parallelism that we've seen in training. While expert GPUs would be processing micro-batch i , attention GPUs would process micro-batch $i+2$, and micro-batch $i+1$ would be enroute from communication delays. This overlapping design hides the communication latency cost while increasing throughput, reduces idle times, and pipeline bubbles.

In addition to pipelining, *MegaScale-infer* introduced a customized M-to-N (attention to experts) communication library to support token routing between attention and GPU nodes. The disaggregation transformed a traditional all-to-all pattern to an inter-node M-to-N. Conventional

libraries like NCCL are optimized for synchronized operations and incur additional overhead for unnecessary features in this use case which can be removed to reduce latency.

Limitations

In scenarios where smaller batch sizes are routed to experts (ex. from small amounts of attention replicas), the benefits of token aggregation may not persist and the overhead from the communication could reduce the overall efficiency of the system. Additionally, ping-pong parallelism is dependent on maintaining balanced time constraints. If the time for computing attention is more than the computing expert node, the pipeline would begin to experience stalls or inefficiencies. Other possible issues like uneven expert routing and network variability can also contribute to these stalls. The M-to-N communication library mitigates much of the issues, however, this is under the assumption that every node is connected with high-bandwidth RDMA interconnects.

Future Research Directions

The authors do not explicitly discuss future directions; any extensions would be reader inferences rather than stated claims.

A possible research avenue that can be explored is expanding upon the deployment planner to be adaptive instead of offline. Future systems could continuously monitor the current workload from latency distributions and automatically adjust micro-batch sizes and attention replications. Another direction would be to improve the fault-tolerance in the disaggregation of attention and expert modules. Since these stages are decoupled but needed to communicate frequently, failures in one can stall the entire pipeline. A failure-aware scheduling and communication algorithm could further strengthen the resilience of the system against faults in large-scale environments.

Summary of Class Discussion

LoongServe

- **Is zero-overhead elastic scaling actually guaranteed, especially for the KV-cache movement?**
 - The paper does not mean zero overhead in the absolute sense. LoongServe's elastic scaling avoids extra KV-cache migration communication in the scaling cases they design for. So it's a narrower systems claim, not a universal guarantee that scaling is free.
- **As workloads change over time, can KV-cache management become less efficient (e.g., due to fragmentation or eviction pressure)?**
 - Yes. Even if the KV values remain valid, KV-cache management can become less efficient over time as workload patterns shift. LoongServe addresses this

with distributed KV-cache management and scheduling that considers memory pressure and future growth.

- **Fault tolerance**
 - The paper does not explore this.
- **How does the system decide how many multi-master instances to assign?**
 - The system may be compute-bound at the master, so adding masters can help.
 - The paper introduces the mechanism, but the selection policy is not fully explicit.
- **Would having multiple master instances improve latency due to splitting parallel groups?**
 - It's not directly discussed in the paper. It's possible to lose some benefits due to more communication overhead, though a single master can become a bottleneck.
 - If there is a global pool of masters where each master manages a subset of the load, there needs to be proper load balancing:
 - Multiple masters would typically have an overlap of the machines they are in charge of.
 - Typically machines depend on multiple masters – an idea close to a global pool of masters.
 - This converts from a compute problem to a load balancing problem.
 - A balls-into-bins framing is useful for this type of load balancing problem (how evenly work is distributed matters a lot).
 - In multi-node settings, memory fragmentation can also become a bottleneck.
 - In a multi-master scenario, each master would also be taking less load.
- **Does having too many degrees of parallelism become a bad thing?**
 - Yes. LoongServe's Elastic Sequence Parallelism (ESP) exists partly because the best DoP depends on request length and serving phase.
 - In decoding (a lighter-weight phase), increasing DoP can give little benefit and may even hurt performance due to communication overhead.
 - Under heavy load, using a lower DoP for requests that scale poorly can be better because it frees resources for other requests.
 - We want to increase parallelism only up to the point where resources are well utilized (compute or memory).
 - Beyond that point, the system can become underutilized/inefficient due to overhead.
 - In queueing terms, there must be a gap between arrival rate and service rate; otherwise, the system can become unstable.
 - Real systems often operate between underutilized and overutilized regions because requests are variable.
 - The system is constantly trying to balance resource usage to control queue length and latency.

MegaScale-Infer

- **How is MegaScale-Infer different from nanobatching or standard pipeline parallelism?**
 - MegaScale-Infer's ping-pong pipeline splits a batch into micro-batches so that attention nodes and expert nodes (disaggregated modules) stay busy and communication is hidden.
 - This is not just ordinary nanobatching on one module. It is a cross-module pipeline (attention and expert) within the MoE serving architecture.
 - It also differs from classical pipeline parallelism that splits model layers into stages.
- **How would it perform in more resource constrained environments with less GPUs**
 - The deployment plan search algorithm would help find the correct parameters for proper scheduling. The paper doesn't have an example of a more resource constrained environment but this could be another item explored later.
 - Even with a large number of GPUs, we plan on smaller block of GPUs and replicate the block over the larger pool of GPUs This makes search and communication more efficient
- **How often are the constraints met on the MegaScale paper, what happens when they're not met?**
 - The paper gives necessary conditions for good overlap:
 - roughly balanced attention/expert compute times,
 - communication shorter than the bottleneck compute time,
 - and enough micro-batches to fill the pipeline.
 - If these fail, communication and dependency stalls are no longer hidden well, so GPU idle time increases and throughput drops